

COMPUTER NETWORKS

Week-11

Checksum - Comments

- Checksum relies on 1's complement arithmetic because it:
- Preserves carries (no loss of information)
- Enables a simple “all 1s” verification rule
- Is efficient and easy to implement
- Works well for typical network errors

Sources of Transmission Errors

- E. Comer
- Chp 8
- RIY

Types of errors

- Single bit Error
- A single bit in a block of bits is changed
- All other bits in the block are unchanged
- Often results from very short-duration interference

Types of errors

- Burst Error
- Multiple bits in a block of bits are changed
- Often results from longer-duration interference

Types of errors

- Erasure (Ambiguity)
- The signal that arrives at a receiver is ambiguous
- Does not clearly correspond to either a logical 1 or a logical 0
- Can result from distortion or interference

Single Parity Checking (SPC)

- One form of SPC defines a block to be an 8-bit unit of data (i.e., a single *byte*)
- On the sending side, an encoder adds an extra bit → *parity bit* to each byte before transmission
- A receiver removes the parity bit and uses it to check whether bits in the byte are correct

Single Parity Checking

- Before parity can be used, the sender and receiver must be configured for either *even parity* or *odd parity*
- When using even parity, the sender chooses a parity bit of 0 if the byte has an even number of 1 bits
- The sender chooses 1 if the byte has an odd number of 1 bits
- Even or odd parity specifies whether the 9 bits sent across a channel have an even or odd number of 1 bits

Data and corresponding value of a single parity bit

Original Data	Even Parity	Odd Parity
0 0 0 0 0 0 0 0	0	1
0 1 0 1 1 0 1 1	1	0
0 1 0 1 0 1 0 1	0	1
1 1 1 1 1 1 1 1	0	1
1 0 0 0 0 0 0 0	1	0
0 1 0 0 1 0 0 1	1	0

SPC

- A weak form of channel coding that can detect errors
- Cannot correct errors
- Can only handle errors where an odd number of bits are changed
- If one of the nine bits (including the parity bit) is
- changed during transmission, the receiver will declare that the incoming byte is **invalid**
- if a burst error occurs in which two, four, six, or eight bits change value, the receiver will incorrectly classify the incoming byte as **valid**

Ex-1

- Given Data
- 10101101
- Total number of 1s?

Ex-1

- Given Data
- 10101101
- Total number of 1s = 5 (Odd)
- If we are using Even parity → total number of 1s should be even
- So we need an additional 1 bit
- Parity bit = 1
- 10101101 1

Ex-1

- Final Transmitted data:
- 10101101 1
- Receiver counts number of 1s
- Total 1s = 6 $\rightarrow$ even
- No error detected

Ex-1

- Lets assume, one bit flipped during transmission:
- Given Data : 10101101
- **Received Data : 101001011**
- Count 1s : 5 → Odd → **Incorrect** / Error detected

Ex-2

- If two bits flipped
- Original : 101011011
- Received : 100011011
- Count 1s : 6 $\rightarrow$ even -- No error detected

Ex-3

- Odd parity example
- Given Data : 10101101
- Count 1s : 5 → Odd
- Parity Bit added : 0
- Transmitted Data : 10101101 0

SPC

- Simplest Error detection method
- Used in simple communication links at data link layer of OSI model
- To detect errors in frames
- Widely used in serial data transmission
- E.g. microcontroller to device communication, embedded systems
- Like a computer communicating with a sensor or modem, Ram error detection
- Too weak for large data transmission

SPC

- Still useful in :
 - IoT Devices
 - Embedded systems
 - Low speed communication links
- Modern networks prefer Checksum and CRC

SPC

- The position of the parity bit depends on the system design, protocol, or teaching convention
- In most computer organization and networking examples, common practice is to add parity bit at the left end (MSB side)
- Keeps original data unchanged
- Easier to treat parity as a “check bit” added externally

SPC

- Sometimes, parity bit is added as LSB
- Easier in serial transmission systems
- Some hardware shifts bits and appends parity at the end

SPC

- Parity bit is not tied to MSB or LSB by theory
- It is simply an extra bit added for error detection
- You must always check the system specification or question instruction

Ex-4 (For Practice)

- A system uses even parity for 16-bit data words
 - The parity bit is added at the LSB side
 - The 16-bit data to be transmitted is: D596
1. Calculate the parity bit
 2. Write the transmitted 17-bit word
 3. Show how the receiver checks correctness

Ex-4 (For Practice)

1. Calculate the parity bit

Data:

1101 0101 1001 0110

count 1s:

1101 → 3 ones

0101 → 2 ones

1001 → 2 ones

0110 → 2 ones

Total:

$3 + 2 + 2 + 2 = 9$ ones

9 is odd

Ex-4 (For Practice)

Current 1s = 9 (odd)

Add parity bit = 1 (to make it even)

Parity bit = 1

Find out the final transmitted frame:

Ex-4 (For Practice)

Find out the final transmitted frame:

1101 0101 001 0110 1

Total no. of 1s : 10 → even → NO error detected

Ex-5 Parity Bit Checking (To be submitted)

- Suppose the sender wants to send the word **PopQuiz**
 - ASCII is applied character by character and parity is applied
1. Convert the following decimal values into binary (to align with 8-bit ASCII codes):
 2. 80 111 112 81 117 105 122
 3. Work out the final data stream in binary
 4. Assume even parity is used for each character on MSB side
 5. Show complete working at the receiver end to decide if correct data has been received or not

Error Handling Strategies

- Mathematical techniques have been developed to overcome data errors
- For better reliability
- Known collectively as *channel coding*
- Two broad categories:
 - Forward Error Correction (FEC) mechanisms
 - Automatic Repeat reQuest (ARQ) mechanisms

Forward Error Correction (FEC)

- Add additional information to data that allows a receiver to verify that data arrives correctly and to **correct errors, if possible**
- Allows a receiver to determine exactly which bits have been changed and to compute correct values

Automatic Repeat reQuest (ARQ)

- Requires the cooperation of a sender
- Sender and receiver exchange messages to ensure that all data arrives correctly

Transmission Errors

- Transmission errors are inevitable
- Error detection mechanisms add overhead
- Need to carefully choose exactly which error detection and compensation mechanisms will be used

Book-B

- Chapter 8
- Douglas E. Comer